

Mathematical Description of the ORCA Family

From Velocity Obstacles to ORCA, S-ORCA, and NH-ORCA
Implementations in `vo.py`, `rvo.py`, `orca.py`, `sorca.py`, and `nhorca.py`

Saulo José (UFCG)

11–13 September 2026

Abstract

This document is the mathematical reference for the three collision-avoidance implementations shipped in `vo.py`, `rvo.py`, `orca.py`, `sorca.py`, and `nhorca.py`. It is written to be read alongside the code: every geometric quantity, every branch of the algorithms, and every numerical convention is derived here and then identified with the function or variable that realizes it.

We begin by motivating the Velocity Obstacle (VO) from the elementary disk-overlap condition, then introduce the Reciprocal Velocity Obstacle (RVO) as the symmetric, multi-agent extension of VO in which both agents share the avoidance effort. We show that RVO is geometrically correct but computationally awkward: the set of admissible velocities is the complement of a union of cones, which is non-convex, and choosing the best velocity inside it is not a convex problem. This motivates ORCA: a construction that replaces the RVO region by a single half-plane per neighbor, tangent to the RVO boundary, and thereby turns velocity selection into a convex quadratic program — the projection of the preferred velocity onto the intersection of a disk and m half-planes. We then explain how the code solves this QP exactly by an incremental sequence of one-dimensional linear programs, with a fallback for infeasible cases. All numerical conventions used in the code (responsibility factors, time horizon τ , neighbor truncation, acceleration limits, angular bias, tolerances) are stated explicitly.

We then extend the framework to differential-drive robots. S-ORCA applies ORCA to an offset *effective center* and inverts the wheel-to-effective velocity map, which is well-posed because the map has determinant D_i/L_i . NH-ORCA closes the theoretical gap left by S-ORCA by deriving a formal bound $\mathcal{E}$ on the tracking error, expanding the effective radius accordingly, and projecting the holonomic velocity onto a convex polygon P_{AHV} contained in the set of trackable velocities. The document is intended as a reference for reproducing, debugging, or extending the implementations.

Contents

Part I: Holonomic ORCA	4
1 Notation and Problem Statement	4
2 The Velocity Obstacle (VO)	4
2.1 Relative position and velocity	4
2.2 Collision condition	5
2.3 Definition of the Velocity Obstacle	5
2.4 Truncated Velocity Obstacle	5
3 The Reciprocal Velocity Obstacle (RVO)	5
3.1 Motivation: why VO is not enough	5
3.2 Definition of RVO	6
3.3 Geometric picture and non-convexity	6
3.4 RVO in the code	6
4 The ORCA Half-Plane	6
4.1 Derivation of the half-plane	7
4.2 Geometric interpretation	7
4.3 Why ORCA is conservative but safe	7
5 Collision Cone Geometry and the Cut-Off Test	8
5.1 The cut-off circle test	8
5.2 Projection onto a cone leg	8
5.3 Immediate collision	9
6 Velocity Selection as a Linear Program	9
6.1 The ORCA optimization problem	9
6.2 1D linear program on a line	9
6.3 2D linear program	10
6.4 3D fallback	10
6.5 Velocity clamping	10
7 Practical Considerations in the Code	10
7.1 Preferred velocity and angular bias	10
7.2 Neighbor selection and truncation	11
7.3 Numerical tolerances	11
7.4 Static obstacles	11
8 Complexity and Convergence	11
9 Summary of the ORCA Algorithm	12
Part II: S-ORCA	12
10 Motivation and Model	12
10.1 Effective state	13
10.2 Effective velocity	13
10.3 Inverse kinematics	13

11 The S-ORCA Algorithm	14
11.1 Goals in the effective frame	14
11.2 Choice of D	14
11.3 Relation to the real robot	14
12 Properties and Limitations of S-ORCA	14
 Part III: NH-ORCA	 15
13 Motivation	15
14 Tracking Error of a Differential-Drive Robot	15
14.1 Setup	15
14.2 Derivation of the squared error	15
14.3 Optimal control for a given holonomic velocity	16
15 The Set S_{AHV} and Its Boundary	16
15.1 Definition	16
15.2 Maximum holonomic speed per direction	16
15.3 Polygon P_{AHV}	17
16 The NH-ORCA Algorithm	17
16.1 Adaptive tracking error (Remark 4)	17
16.2 Radial projection onto P_{AHV}	18
16.3 Comparison with S-ORCA	18
17 Summary of the Extended Algorithm Family	18
References	19

Part I: Holonomic ORCA

1 Notation and Problem Statement

We consider a planar multi-agent system of N disk-shaped agents moving in $\mathbb{R}^2$. Agent i has

- position $\mathbf{p}_i \in \mathbb{R}^2$,
- velocity $\mathbf{v}_i \in \mathbb{R}^2$,
- radius $r_i > 0$,
- goal $\mathbf{g}_i \in \mathbb{R}^2$,
- preferred velocity $\mathbf{v}_i^{\text{pref}} \in \mathbb{R}^2$,

and is subject to the kinematic constraints

$$\|\mathbf{v}_i\| \leq v_{\max}, \quad \|\mathbf{v}_i^{\text{new}} - \mathbf{v}_i\| \leq a_{\max} \Delta t, \quad (1)$$

where $v_{\max}$ is the maximum speed, $a_{\max}$ the maximum acceleration, and Δt the simulation time step. The agents are updated synchronously: at each step every agent computes a new velocity $\mathbf{v}_i^{\text{new}}$ using the current positions and velocities of its neighbors, and then integrates $\mathbf{p}_i \leftarrow \mathbf{p}_i + \mathbf{v}_i^{\text{new}} \Delta t$.

A static obstacle o is modeled as an agent with $\mathbf{v}_o = \mathbf{0}$ and radius r_o , for which the ego agent assumes *full* responsibility for avoidance.

Collision horizon. Throughout Part I, $\tau > 0$ denotes the *collision horizon*: the time window over which future collisions are predicted. Velocities that would only cause a collision after τ are treated as safe. This is the horizon used in the ORCA paper and in the RVO2 library; it is the same symbol in every section below.

Assumptions.

1. Perfect sensing: all neighbors within $d_{\max}$ are observed.
2. Homogeneous decision model: every reciprocal agent uses the same algorithm with the same parameters. This is what makes the half-planes symmetric and the avoidance reciprocal.
3. No dynamics beyond (1): the next velocity is chosen freely inside the admissible disk, then rate-limited.

2 The Velocity Obstacle (VO)

The starting point of the entire ORCA family is a purely geometric construction: for a pair of disk-shaped agents, describe *which velocities lead to a collision* and *which do not*. This is the Velocity Obstacle.

2.1 Relative position and velocity

For a pair (a, b) define

$$\mathbf{p}_{ab} := \mathbf{p}_b - \mathbf{p}_a, \quad \mathbf{v}_{ab} := \mathbf{v}_a - \mathbf{v}_b, \quad R_{ab} := r_a + r_b. \quad (2)$$

Here $\mathbf{p}_{ab}$ points from a to b , $\mathbf{v}_{ab}$ is the velocity of a *relative to* b , and R_{ab} is the sum of radii (Minkowski sum of the two disks). Working in relative coordinates is the key simplification: the two-agent avoidance problem becomes a single-agent problem in which the origin represents b and a moves with velocity $\mathbf{v}_{ab}$.

2.2 Collision condition

The two disks overlap at time $t \geq 0$ if

$$\|\mathbf{p}_{ab} + t \mathbf{v}_{ab}\| \leq R_{ab}. \quad (3)$$

This is simply the statement that the distance between the two centers, after a has moved by $t \mathbf{v}_a$ and b by $t \mathbf{v}_b$, is at most the sum of the radii. In relative coordinates it is exact: the relative position after time t is $\mathbf{p}_{ab} + t \mathbf{v}_{ab}$.

2.3 Definition of the Velocity Obstacle

We adopt the convention of the ORCA paper: the Velocity Obstacle VO_{ab}^τ is a set of *absolute* velocities $\mathbf{v}_a$ of agent a , parameterized by the current velocity $\mathbf{v}_b$ of agent b :

$$\text{VO}_{ab}^\tau(\mathbf{v}_b) = \left\{ \mathbf{v}_a \in \mathbb{R}^2 : \mathbf{v}_a - \mathbf{v}_b \in \text{CC}_{ab}^\tau \right\} = \mathbf{v}_b + \text{CC}_{ab}^\tau, \quad (4)$$

where CC_{ab}^τ is the *collision cone* of relative velocities,

$$\text{CC}_{ab}^\tau = \left\{ \mathbf{v} \in \mathbb{R}^2 : \exists t \in (0, \tau], \|\mathbf{p}_{ab} + t \mathbf{v}\| \leq R_{ab} \right\}. \quad (5)$$

The set CC_{ab}^τ is a truncated cone with apex at the origin of relative velocity space, axis along $-\mathbf{p}_{ab}$, and half-angle

$$\alpha_{ab} = \arcsin\left(\frac{R_{ab}}{\|\mathbf{p}_{ab}\|}\right), \quad \|\mathbf{p}_{ab}\| > R_{ab}. \quad (6)$$

The truncation at time τ cuts the cone at distance $\|\mathbf{p}_{ab}\|/\tau$ along the axis, giving a flat circular cap of radius R_{ab}/τ . When $\|\mathbf{p}_{ab}\| \leq R_{ab}$ the disks already overlap, the cone degenerates to the whole plane, and the case is handled separately in Section 5.3.

2.4 Truncated Velocity Obstacle

Without truncation, (5) declares a collision even if contact occurs in the far future. Truncation at τ is what makes the obstacle computationally finite: velocities that only become dangerous after τ are treated as safe. Throughout this document τ is the same collision horizon, and the superscript τ is dropped only when the context is unambiguous.

3 The Reciprocal Velocity Obstacle (RVO)

The Velocity Obstacle is defined under the assumption that *only agent a changes its velocity*; agent b keeps $\mathbf{v}_b$ fixed. In a multi-agent setting this is unfair and oscillatory: if both agents use VO naively, each treats the other as a moving obstacle, and they can chase each other or freeze. The Reciprocal Velocity Obstacle removes this asymmetry by requiring that *both* agents share the avoidance effort.

3.1 Motivation: why VO is not enough

Suppose two agents are on a head-on collision course. Under the standard VO framework, agent a selects a new velocity assuming $\mathbf{v}_b$ remains constant, while agent b simultaneously does the same assuming $\mathbf{v}_a$ is fixed. Because VO ignores the reciprocal nature of the interaction, both agents continuously update their velocities in response to one another. This causes *reciprocal chatter*: agents oscillate between evasive maneuvers and their preferred velocities, producing unstable trajectories. The VO is a *one-sided* model, and one-sided models fail to compose in reciprocal multi-agent settings.

3.2 Definition of RVO

Fix a time horizon τ and consider a pair (a, b) with current velocities $(\mathbf{v}_a, \mathbf{v}_b)$. The *Reciprocal Velocity Obstacle* RVO_{ab}^τ is the set of *new* velocities $\mathbf{v}_a^{\text{new}}$ for agent a such that, if agent b makes an equal and opposite effort in relative velocity space, the pair collides within time horizon τ :

$$\text{RVO}_{ab}^\tau = \left\{ \mathbf{v}_a^{\text{new}} \in \mathbb{R}^2 : 2\mathbf{v}_a^{\text{new}} - \mathbf{v}_a \in \text{VO}_{ab}^\tau(\mathbf{v}_b) \right\}. \quad (7)$$

Equivalently, under the reciprocal assumption $\mathbf{v}_b^{\text{new}} - \mathbf{v}_b = -(\mathbf{v}_a^{\text{new}} - \mathbf{v}_a)$, the resulting relative velocity is

$$\mathbf{v}_{ab}^{\text{new}} = \mathbf{v}_a^{\text{new}} - \mathbf{v}_b^{\text{new}} = (\mathbf{v}_a - \mathbf{v}_b) + 2(\mathbf{v}_a^{\text{new}} - \mathbf{v}_a). \quad (8)$$

The pair collides if $\mathbf{v}_{ab}^{\text{new}} \in \text{CC}_{ab}^\tau$, which expands to

$$(\mathbf{v}_a - \mathbf{v}_b) + 2\mathbf{v}_a^{\text{new}} - 2\mathbf{v}_a \in \text{CC}_{ab}^\tau \iff 2\mathbf{v}_a^{\text{new}} - \mathbf{v}_a \in \mathbf{v}_b + \text{CC}_{ab}^\tau = \text{VO}_{ab}^\tau(\mathbf{v}_b). \quad (9)$$

Geometrically, RVO_{ab}^τ is obtained by shifting the apex of the collision cone to the midpoint of the two current velocities and scaling its bounds by $\frac{1}{2}$:

$$\text{RVO}_{ab}^\tau = \frac{1}{2} (\mathbf{v}_a + \text{VO}_{ab}^\tau(\mathbf{v}_b)) = \frac{\mathbf{v}_a + \mathbf{v}_b}{2} + \frac{1}{2} \text{CC}_{ab}^\tau. \quad (10)$$

This is the apex form used in the RVO paper: the forbidden cone is anchored at $(\mathbf{v}_a + \mathbf{v}_b)/2$, not at $\mathbf{v}_a$.

3.3 Geometric picture and non-convexity

The RVO is a truncated cone whose apex is at $(\mathbf{v}_a + \mathbf{v}_b)/2$, pointing away from the other agent. The set of *safe* velocities is the complement of the RVO inside the admissibility disk. With m neighbors,

$$\mathcal{S} = \left\{ \mathbf{v} \in \mathbb{R}^2 : \|\mathbf{v}\| \leq v_{\max} \right\} \setminus \bigcup_{j=1}^m \text{RVO}_{a|j}^\tau. \quad (11)$$

This set is *non-convex*: it is a disk with m truncated cones subtracted. Choosing the velocity in $\mathcal{S}$ closest to $\mathbf{v}_a^{\text{pref}}$ is therefore a non-convex optimization problem. This is the fundamental obstacle that ORCA removes.

3.4 RVO in the code

The implementation does not compute the full RVO cone; it computes only the two geometric quantities $(\mathbf{u}, \mathbf{n})$ that characterize the closest boundary of the cone to a reference relative velocity, as in Sections 5–5.2. These are the quantities from which both RVO and ORCA half-planes can be built. The distinction between RVO and ORCA is entirely in how $(\mathbf{u}, \mathbf{n})$ are used:

- **RVO** uses the full cone RVO_{ab}^τ as the forbidden region; feasible velocities are the complement (11), which is non-convex.
- **ORCA** uses only the half-plane tangent to the cone at the point closest to the reference velocity. Feasible velocities are the intersection of half-planes, which is convex.

This is the entire conceptual leap from RVO to ORCA.

4 The ORCA Half-Plane

ORCA is the observation that one can replace the non-convex RVO region by a single half-plane per neighbor, chosen tangent to the RVO cone, without losing the collision-avoidance guarantee — provided the tangent point is chosen correctly. The result is a convex feasible set and a fast, closed-form velocity-selection step.

4.1 Derivation of the half-plane

Consider a pair (a, b) and the collision cone CC_{ab}^τ of (5). Let $\mathbf{v}_a^{\text{opt}}$ and $\mathbf{v}_b^{\text{opt}}$ be the *reference* velocities used to define the half-plane. In the ORCA paper, $\mathbf{v}_a^{\text{opt}}$ is the preferred velocity; in the RVO2 implementation, $\mathbf{v}_a^{\text{opt}}$ is the current velocity. Both choices are valid; in the code we use the current velocity, so $\mathbf{v}_a^{\text{opt}} = \mathbf{v}_a$ and $\mathbf{v}_b^{\text{opt}} = \mathbf{v}_b$ throughout.

Define the current relative velocity and relative position

$$\mathbf{v}_{ab}^{\text{opt}} := \mathbf{v}_a^{\text{opt}} - \mathbf{v}_b^{\text{opt}}, \quad \mathbf{p}_{ab} := \mathbf{p}_b - \mathbf{p}_a. \quad (12)$$

Let $\mathbf{u}$ be the smallest vector that pushes $\mathbf{v}_{ab}^{\text{opt}}$ out of the collision cone CC_{ab}^τ :

$$\mathbf{u} = \left(\arg \min_{\mathbf{w} \in \partial \text{CC}_{ab}^\tau} \|\mathbf{w} - \mathbf{v}_{ab}^{\text{opt}}\| \right) - \mathbf{v}_{ab}^{\text{opt}}, \quad (13)$$

and let $\mathbf{n} = \mathbf{u}/\|\mathbf{u}\|$ be the outward unit normal at the projection point. The supporting half-plane of the cone is then

$$\text{CC}_{ab}^\tau \subseteq \left\{ \mathbf{v} \in \mathbb{R}^2 : (\mathbf{v} - \mathbf{v}_{ab}^{\text{opt}} - \mathbf{u}) \cdot \mathbf{n} < 0 \right\}, \quad (14)$$

i.e. the half-plane $\{\mathbf{v} : (\mathbf{v} - \mathbf{v}_{ab}^{\text{opt}} - \mathbf{u}) \cdot \mathbf{n} \geq 0\}$ lies *outside* the cone and touches it at $\mathbf{v}_{ab}^{\text{opt}} + \mathbf{u}$. The reciprocal responsibility split assigns half of the correction to each agent, so agent a 's half-plane is anchored at $\mathbf{v}_a^{\text{opt}} + \frac{1}{2}\mathbf{u}$:

$$\text{ORCA}_{a|b}^\tau = \left\{ \mathbf{v} \in \mathbb{R}^2 : \left(\mathbf{v} - \left(\mathbf{v}_a^{\text{opt}} + \frac{1}{2}\mathbf{u} \right) \right) \cdot \mathbf{n} \geq 0 \right\}. \quad (15)$$

For a static obstacle the factor $\frac{1}{2}$ is replaced by 1, because the obstacle cannot move and the ego agent must absorb the entire correction:

$$\text{ORCA}_{a|o}^\tau = \left\{ \mathbf{v} \in \mathbb{R}^2 : \left(\mathbf{v} - \left(\mathbf{v}_a^{\text{opt}} + \mathbf{u} \right) \right) \cdot \mathbf{n} \geq 0 \right\}. \quad (16)$$

Responsibility factor. In the code,

$$w_{\text{resp}} = \begin{cases} 1.0, & \text{static obstacle,} \\ 0.5, & \text{reciprocal agent.} \end{cases} \quad (17)$$

The boundary point is $\mathbf{p}_0 = \mathbf{v}_a^{\text{opt}} + w_{\text{resp}} \mathbf{u}$, and each ORCA constraint is a half-plane

$$(\mathbf{v} - \mathbf{p}_0) \cdot \mathbf{n} \geq 0 \quad \Longleftrightarrow \quad \mathbf{n} \cdot \mathbf{v} \geq \mathbf{n} \cdot \mathbf{p}_0. \quad (18)$$

4.2 Geometric interpretation

The ORCA half-plane (18) is the set of velocities on the “safe” side of the line through $\mathbf{p}_0$ with normal $\mathbf{n}$. For a reciprocal pair, both agents compute their own half-plane using the same $\mathbf{u}$ and $\mathbf{n}$ but opposite responsibility signs, so the two half-planes are mirror images and the pair separates symmetrically. For a static obstacle, the ego agent absorbs the entire correction $\mathbf{u}$.

4.3 Why ORCA is conservative but safe

By construction, the ORCA half-plane is *contained in* the safe set of the RVO: every velocity in $\text{ORCA}_{a|b}^\tau$ is also outside RVO_{ab}^τ , because the half-plane is a supporting half-plane of the cone. The converse is not true: some velocities outside the RVO may fail the ORCA half-plane test. ORCA is therefore *conservative* — it forbids a slightly larger set of velocities than strictly necessary — but never unsafe with respect to the RVO. The conservatism is the price paid for convexity, and it is what makes ORCA tractable.

5 Collision Cone Geometry and the Cut-Off Test

We now derive the concrete formulas for $(\mathbf{u}, \mathbf{n})$ used in the code. These formulas are shared by VO, RVO, and ORCA; only the way they are combined differs.

5.1 The cut-off circle test

Define the auxiliary vector

$$\mathbf{w} := \mathbf{v}_{ab}^{\text{opt}} - \frac{\mathbf{p}_{ab}}{\tau}, \quad (19)$$

whose tip, drawn from the apex of the truncated cone, points to the current relative velocity. The point on the truncated cone boundary closest to $\mathbf{v}_{ab}^{\text{opt}}$ lies either on the circular cap (the “cut-off disk”) or on one of the two straight legs. The code first tests whether it lies on the cap, using the two conditions

$$\mathbf{p}_{ab} \cdot \mathbf{w} < 0 \quad \text{and} \quad (\mathbf{p}_{ab} \cdot \mathbf{w})^2 > R_{ab}^2 \|\mathbf{w}\|^2. \quad (20)$$

The two conditions have a clean geometric meaning.

- $\mathbf{p}_{ab} \cdot \mathbf{w} < 0$ requires the relative velocity to be directed *toward* the other agent. If $\mathbf{p}_{ab} \cdot \mathbf{w} \geq 0$, the pair is separating and no collision is possible within τ .
- $(\mathbf{p}_{ab} \cdot \mathbf{w})^2 > R_{ab}^2 \|\mathbf{w}\|^2$ is equivalent to $|\mathbf{p}_{ab} \cdot \hat{\mathbf{w}}| > R_{ab}$: the projection of the relative position onto the direction of $\mathbf{w}$ exceeds the combined radius, so the closest boundary point of the cone lies on the circular cap rather than on a leg. This is the canonical test used in the RVO2 library and in the ORCA reference implementation.

When both conditions hold, the smallest correction is a radial displacement to the cap boundary:

$$\mathbf{u} = \left(\frac{R_{ab}}{\tau} - \|\mathbf{w}\| \right) \frac{\mathbf{w}}{\|\mathbf{w}\|}, \quad \mathbf{n} = \frac{\mathbf{w}}{\|\mathbf{w}\|}. \quad (21)$$

5.2 Projection onto a cone leg

If (20) does not hold, the closest point on the cone boundary is on one of the two legs. Let

$$\ell := \sqrt{\|\mathbf{p}_{ab}\|^2 - R_{ab}^2} \quad (22)$$

be the length of the tangent from the origin of relative velocity space to the Minkowski disk of radius R_{ab} . The two unit directions along the left and right cone legs are

$$\mathbf{d}_L = \frac{1}{\|\mathbf{p}_{ab}\|^2} \begin{pmatrix} \mathbf{p}_{ab,x} \ell - \mathbf{p}_{ab,y} R_{ab} \\ \mathbf{p}_{ab,x} R_{ab} + \mathbf{p}_{ab,y} \ell \end{pmatrix}, \quad (23)$$

$$\mathbf{d}_R = \frac{1}{\|\mathbf{p}_{ab}\|^2} \begin{pmatrix} -\mathbf{p}_{ab,x} \ell - \mathbf{p}_{ab,y} R_{ab} \\ \mathbf{p}_{ab,x} R_{ab} - \mathbf{p}_{ab,y} \ell \end{pmatrix}. \quad (24)$$

The sign of the cross product

$$\det(\mathbf{p}_{ab}, \mathbf{w}) = \mathbf{p}_{ab,x} \mathbf{w}_y - \mathbf{p}_{ab,y} \mathbf{w}_x \quad (25)$$

selects which leg is closer: if $\det > 0$, the left leg $\mathbf{d}_L$ is used; otherwise the right leg $\mathbf{d}_R$. The closest point on the chosen leg is the orthogonal projection of $\mathbf{v}_{ab}^{\text{opt}}$ onto the line spanned by $\mathbf{d}$:

$$\mathbf{u} = (\mathbf{v}_{ab}^{\text{opt}} \cdot \mathbf{d}) \mathbf{d} - \mathbf{v}_{ab}^{\text{opt}}, \quad \mathbf{n} = (-\mathbf{d}_y, \mathbf{d}_x). \quad (26)$$

Note that $\mathbf{n}$ is perpendicular to $\mathbf{d}$ and points *outside* the cone.

5.3 Immediate collision

If $\|\mathbf{p}_{ab}\| \leq R_{ab}$, the disks already overlap. The code replaces τ by Δt in (19)–(21):

$$\mathbf{w} = \mathbf{v}_{ab}^{\text{opt}} - \frac{\mathbf{p}_{ab}}{\Delta t}, \quad \mathbf{u} = \left(\frac{R_{ab}}{\Delta t} - \|\mathbf{w}\| \right) \frac{\mathbf{w}}{\|\mathbf{w}\|}, \quad \mathbf{n} = \frac{\mathbf{w}}{\|\mathbf{w}\|}. \quad (27)$$

The horizon τ is normally a prediction window; when the disks already overlap, the only meaningful horizon is the immediate step Δt , and the correction is computed as if the collision had to be resolved within a single time step. The effect is to force immediate separation.

6 Velocity Selection as a Linear Program

6.1 The ORCA optimization problem

At each time step, agent a collects the ORCA half-planes of all neighbors within $d_{\max}$:

$$\mathcal{H} = \{(\mathbf{p}_{0,i}, \mathbf{n}_i)\}_{i=1}^m, \quad \mathbf{n}_i \cdot \mathbf{v} \geq \mathbf{n}_i \cdot \mathbf{p}_{0,i}. \quad (28)$$

It then solves

$$\begin{aligned} \mathbf{v}_a^{\text{new}} &= \arg \min_{\mathbf{v} \in \mathbb{R}^2} \|\mathbf{v} - \mathbf{v}_a^{\text{pref}}\|^2 \\ \text{s.t.} \quad &\|\mathbf{v}\| \leq v_{\max}, \\ &\mathbf{n}_i \cdot \mathbf{v} \geq \mathbf{n}_i \cdot \mathbf{p}_{0,i}, \quad i = 1, \dots, m. \end{aligned} \quad (29)$$

This is a convex quadratic program: the objective is strictly convex, the feasible set is the intersection of a disk and m half-planes, and the intersection of convex sets is convex. The optimal solution is the Euclidean projection of $\mathbf{v}_a^{\text{pref}}$ onto that intersection.

6.2 1D linear program on a line

Consider the line

$$\mathcal{L}_i : \mathbf{v}(t) = \mathbf{p}_{0,i} + t \mathbf{d}_i, \quad \mathbf{d}_i = (-\mathbf{n}_{i,y}, \mathbf{n}_{i,x}), \quad (30)$$

which is the boundary of constraint i . The disk constraint $\|\mathbf{v}(t)\| \leq v_{\max}$ becomes

$$t^2 + 2(\mathbf{p}_{0,i} \cdot \mathbf{d}_i) t + \|\mathbf{p}_{0,i}\|^2 - v_{\max}^2 \leq 0, \quad (31)$$

so the feasible interval is

$$t \in [t_{\min}, t_{\max}], \quad t_{\min/\max} = -(\mathbf{p}_{0,i} \cdot \mathbf{d}_i) \pm \sqrt{v_{\max}^2 - (\mathbf{p}_{0,i} \cdot \mathbf{n}_i)^2}, \quad (32)$$

provided the discriminant is nonnegative. The term $\mathbf{p}_{0,i} \cdot \mathbf{n}_i$ is the signed distance from the origin to the line $\mathcal{L}_i$; if it exceeds $v_{\max}$ in absolute value, the line does not intersect the disk and the 1D LP is infeasible. Each previously accepted constraint $j < i$ becomes a scalar inequality in t :

$$(\mathbf{p}_{0,i} + t \mathbf{d}_i - \mathbf{p}_{0,j}) \cdot \mathbf{n}_j \geq 0 \iff A_j + t B_j \geq 0, \quad (33)$$

with

$$A_j = (\mathbf{p}_{0,i} - \mathbf{p}_{0,j}) \cdot \mathbf{n}_j, \quad B_j = \mathbf{d}_i \cdot \mathbf{n}_j. \quad (34)$$

If $B_j > 0$, then $t \geq -A_j/B_j$; if $B_j < 0$, then $t \leq -A_j/B_j$; if $B_j = 0$, the constraint is redundant or infeasible depending on the sign of A_j . Intersecting all these intervals yields $[t_{\min}, t_{\max}]$. The optimal t is the projection of the preferred velocity onto the line,

$$t^{\text{pref}} = (\mathbf{v}_a^{\text{pref}} - \mathbf{p}_{0,i}) \cdot \mathbf{d}_i, \quad t^* = \text{clamp}(t^{\text{pref}}, t_{\min}, t_{\max}), \quad (35)$$

and the resulting velocity is $\mathbf{v}(t^*)$.

6.3 2D linear program

The 2D LP is solved incrementally. Start with

$$\mathbf{v}^{(0)} = \begin{cases} \mathbf{v}_a^{\text{pref}}, & \|\mathbf{v}_a^{\text{pref}}\| \leq v_{\max}, \\ v_{\max} \frac{\mathbf{v}_a^{\text{pref}}}{\|\mathbf{v}_a^{\text{pref}}\|}, & \text{otherwise.} \end{cases} \quad (36)$$

For each constraint $i = 1, \dots, m$, test $(\mathbf{v}^{(i-1)} - \mathbf{p}_{0,i}) \cdot \mathbf{n}_i \geq 0$. If it passes, keep $\mathbf{v}^{(i)} = \mathbf{v}^{(i-1)}$. If it fails, solve the 1D LP on line i with all previous constraints; if feasible, set $\mathbf{v}^{(i)}$ to its solution; otherwise the 2D LP is infeasible at line i , and the fallback of Section 6.4 is invoked. This is the standard randomized incremental LP scheme of RVO2.

6.4 3D fallback

If the 2D LP fails at line k , the code solves a reduced LP in the subspace spanned by the violated line and each previously accepted line. For each $j < k$, let

$$D_{kj} = \mathbf{n}_k \times \mathbf{n}_j = \mathbf{n}_{k,x} \mathbf{n}_{j,y} - \mathbf{n}_{k,y} \mathbf{n}_{j,x} \quad (37)$$

be the 2D cross product of the two normals, taken in the order (k, j) . If $|D_{kj}| > \varepsilon$, the two lines intersect at

$$\mathbf{p}_{kj} = \mathbf{p}_{0,k} + t_{kj} \mathbf{d}_k, \quad t_{kj} = \frac{(\mathbf{p}_{0,j} - \mathbf{p}_{0,k}) \cdot \mathbf{n}_j}{D_{kj}}. \quad (38)$$

If the lines are parallel and same-oriented, the constraint is redundant; if anti-parallel, the midpoint $\frac{1}{2}(\mathbf{p}_{0,j} + \mathbf{p}_{0,k})$ is used. The projected normal is

$$\mathbf{n}_{kj} = \frac{\mathbf{n}_j - \mathbf{n}_k}{\|\mathbf{n}_j - \mathbf{n}_k\|}. \quad (39)$$

These projected constraints define a 2D LP in the subspace, solved with the same routine. The code updates the residual violation $\delta = (\mathbf{v} - \mathbf{p}_{0,k}) \cdot \mathbf{n}_k$ and keeps the best solution found. The fallback is needed because the intersection of many half-planes with a disk can be empty, for instance when an agent is surrounded by neighbors pressing inward from all sides.

6.5 Velocity clamping

After the LP, the new velocity is rate-limited by (1):

$$\mathbf{v}_a^{\text{new}} \leftarrow \begin{cases} \mathbf{v}_a + a_{\max} \Delta t \frac{\mathbf{v}_a^{\text{new}} - \mathbf{v}_a}{\|\mathbf{v}_a^{\text{new}} - \mathbf{v}_a\|}, & \|\mathbf{v}_a^{\text{new}} - \mathbf{v}_a\| > a_{\max} \Delta t, \\ \mathbf{v}_a^{\text{new}}, & \text{otherwise.} \end{cases} \quad (40)$$

The clamp is applied last, after the LP, so it can in principle reintroduce a small violation of the half-planes; this is acceptable because the half-planes are a predictive safety margin, not an instantaneous hard constraint.

7 Practical Considerations in the Code

7.1 Preferred velocity and angular bias

The preferred velocity is chosen as

$$\mathbf{v}_a^{\text{pref}} = v_{\max} \frac{\mathbf{g}_a - \mathbf{p}_a}{\|\mathbf{g}_a - \mathbf{p}_a\|} \quad (41)$$

rotated by an angle

$$\theta = \theta_{\text{bias}} + \theta_{\text{noise}}, \quad \theta_{\text{noise}} = 0.02 \sin(12.9898 x_a + 78.233 y_a), \quad (42)$$

where θ_{bias} is a small constant (default 0.25 rad) and the noise term breaks the symmetry of head-on encounters. Symmetric deadlocks are a well-known failure mode of ORCA. The bias is scaled down with density:

$$\theta_{\text{bias}} = \text{clip}\left(\frac{\theta_0}{\sqrt{N/10}}, 0.03, \theta_0\right). \quad (43)$$

7.2 Neighbor selection and truncation

Neighbors are found with a k -d tree within radius d_{max} . If more than K neighbors are found (default $K = 10$), only the K closest are kept:

$$\mathcal{N}_a = \arg \min_{S \subseteq \mathcal{N}_a^{\text{raw}}, |S|=K} \sum_{j \in S} \|\mathbf{p}_j - \mathbf{p}_a\|^2. \quad (44)$$

7.3 Numerical tolerances

The code uses $\varepsilon = 10^{-9}$ for detecting degenerate cases. These tolerances are necessary because floating-point arithmetic can produce small nonzero values where exact arithmetic would give zero.

7.4 Static obstacles

Static obstacles are appended to the agent list with zero velocity and a boolean flag marking them as non-reciprocal. The ORCA construction uses $w_{\text{resp}} = 1$ for them, as in (16).

8 Complexity and Convergence

Per-agent cost. For an agent with m effective neighbors, the 2D LP performs $O(m)$ half-plane tests, each in $O(1)$ time. If the LP fails, the 3D fallback performs $O(m^2)$ intersection tests. In practice $m \leq K = 10$.

Global cost. With N agents, the k -d tree query costs $O(N \log N)$ per step, and the per-agent LP adds $O(NK)$ (or $O(NK^2)$ in the worst case).

Convergence. ORCA is not guaranteed to reach the goal in finite time in arbitrary environments (deadlocks are possible), but it guarantees collision-free motion under the following conditions:

1. all agents use the same τ and v_{max} ;
2. the LP is feasible at every step;
3. the time step Δt is small enough that the acceleration limit does not invalidate the half-plane constraints.

9 Summary of the ORCA Algorithm

Algorithm 1 ORCA velocity selection for agent a

```

1: Input:  $\mathbf{p}_a, \mathbf{v}_a, r_a, \mathbf{g}_a$ ; neighbors  $\{(\mathbf{p}_j, \mathbf{v}_j, r_j, \text{is\_obs}_j)\}$ ; parameters
    $\Delta t, v_{\max}, a_{\max}, \tau, d_{\max}, \theta_{\text{bias}}$ .
2: Compute  $\mathbf{v}_a^{\text{pref}}$  from (41)–(43).
3: Initialize empty list of ORCA lines  $\mathcal{H}$ .
4: for all neighbors  $j$  with  $\|\mathbf{p}_j - \mathbf{p}_a\| < d_{\max}$  do
5:    $(\mathbf{p}_{0,j}, \mathbf{n}_j) \leftarrow \text{COMPUTEORCALINE}(a, j)$  ▷ Eqs. (21)–(??)
6:   Append  $(\mathbf{p}_{0,j}, \mathbf{n}_j)$  to  $\mathcal{H}$ .
7: end for
8:  $\mathbf{v}_a^{\text{new}} \leftarrow \text{SOLVELP}(\mathcal{H}, v_{\max}, \mathbf{v}_a^{\text{pref}})$  ▷ Section 6
9: Clamp  $\mathbf{v}_a^{\text{new}}$  by (40).
10: return  $\mathbf{v}_a^{\text{new}}$ 

```

ComputeORCALine.

1. Set $\mathbf{p}_{ab} = \mathbf{p}_j - \mathbf{p}_a$, $\mathbf{v}_{ab}^{\text{opt}} = \mathbf{v}_a - \mathbf{v}_j$, $R = r_a + r_j$, $w_{\text{resp}} = 1$ if j is an obstacle else 0.5.
2. If $\|\mathbf{p}_{ab}\| \leq R$, use (27) with Δt .
3. Else compute $\mathbf{w}$ by (19).
4. If (20) holds, use (21).
5. Else use (22)–(26).
6. Return $\mathbf{p}_0 = \mathbf{v}_a^{\text{opt}} + w_{\text{resp}}\mathbf{u}$ and $\mathbf{n}$.

SolveLP.

1. Clip $\mathbf{v}_a^{\text{pref}}$ to the disk of radius $v_{\max}$ by (36).
2. For each line i , if the current velocity violates it, solve the 1D LP (32)–(35). If the 1D LP fails, run the 3D fallback of Section 6.4.

Part II: S-ORCA (Non-Holonomic ORCA via Effective Center)

10 Motivation and Model

The original ORCA algorithm assumes *holonomic* agents, i.e. agents that can instantaneously change their velocity in any direction. Differential-drive robots are *non-holonomic*: they cannot move sideways. Their state is (x, y, θ) with body-frame linear velocity v and angular velocity ω , and the wheel velocities (v_l, v_r) are related to (v, ω) by

$$v = \frac{v_l + v_r}{2}, \quad \omega = \frac{v_r - v_l}{L}, \quad (45)$$

where L is the wheel base. The non-holonomic constraint is

$$\dot{x} \sin \theta - \dot{y} \cos \theta = 0. \quad (46)$$

S-ORCA (Snape, van den Berg, Guy, and Manocha, 2010) sidesteps the non-holonomic constraint by applying standard ORCA to a *virtual effective center* located at a distance D in front of the robot along its heading, and using an *effective radius* $r + D$. The effective center is fully controllable in velocity space: the map from wheel velocities to effective velocities is invertible, so ORCA can be applied there exactly.

10.1 Effective state

For robot i with real center $\mathbf{p}_i = (x_i, y_i)$, heading θ_i , and real radius r_i , define the effective center

$$\mathbf{p}_i^{\text{eff}} = \mathbf{p}_i + D_i \begin{pmatrix} \cos \theta_i \\ \sin \theta_i \end{pmatrix}, \quad D_i = \kappa r_i, \quad (47)$$

where κ is the `D_scale` parameter (default 1.0). The effective radius is

$$R_i^{\text{eff}} = r_i + D_i. \quad (48)$$

The choice of $R_i^{\text{eff}} = r_i + D_i$ ensures that the real disk is contained in the effective disk regardless of the heading.

10.2 Effective velocity

The velocity of the effective center is obtained by differentiating (47):

$$\dot{\mathbf{p}}_i^{\text{eff}} = \dot{\mathbf{p}}_i + D_i \dot{\theta}_i \begin{pmatrix} -\sin \theta_i \\ \cos \theta_i \end{pmatrix}. \quad (49)$$

Substituting the differential-drive forward kinematics $\dot{x} = v \cos \theta$, $\dot{y} = v \sin \theta$, $\dot{\theta} = \omega$, and using (45), the components become

$$\dot{X}_i = \left(\frac{\cos \theta_i}{2} + \frac{D_i \sin \theta_i}{L_i} \right) v_{l,i} + \left(\frac{\cos \theta_i}{2} - \frac{D_i \sin \theta_i}{L_i} \right) v_{r,i}, \quad (50)$$

$$\dot{Y}_i = \left(\frac{\sin \theta_i}{2} - \frac{D_i \cos \theta_i}{L_i} \right) v_{l,i} + \left(\frac{\sin \theta_i}{2} + \frac{D_i \cos \theta_i}{L_i} \right) v_{r,i}. \quad (51)$$

In matrix form,

$$\begin{pmatrix} \dot{X}_i \\ \dot{Y}_i \end{pmatrix} = M_i(\theta_i) \begin{pmatrix} v_{l,i} \\ v_{r,i} \end{pmatrix}, \quad M_i(\theta) = \begin{pmatrix} \frac{c}{2} + \frac{D_i s}{L_i} & \frac{c}{2} - \frac{D_i s}{L_i} \\ \frac{s}{2} - \frac{D_i c}{L_i} & \frac{s}{2} + \frac{D_i c}{L_i} \end{pmatrix}, \quad (52)$$

with $c = \cos \theta$, $s = \sin \theta$. The determinant is

$$\det M_i = \frac{D_i}{L_i}, \quad (53)$$

nonzero for $D_i > 0$, so M_i is invertible. This is the essential fact that makes S-ORCA well-posed: the map from wheel velocities to effective velocity is a bijection.

10.3 Inverse kinematics

Given a desired effective velocity $\mathbf{v}_i^{\text{eff}} = (\dot{X}_i, \dot{Y}_i)$ from ORCA, the wheel velocities are recovered from the inverse of (52):

$$\begin{pmatrix} v_{l,i} \\ v_{r,i} \end{pmatrix} = M_i(\theta_i)^{-1} \begin{pmatrix} \dot{X}_i \\ \dot{Y}_i \end{pmatrix}. \quad (54)$$

Explicitly,

$$M_i^{-1} = \frac{L_i}{D_i} \begin{pmatrix} \frac{s}{2} + \frac{D_i c}{L_i} & -\frac{c}{2} + \frac{D_i s}{L_i} \\ -\frac{s}{2} + \frac{D_i c}{L_i} & \frac{c}{2} + \frac{D_i s}{L_i} \end{pmatrix}. \quad (55)$$

This is the map implemented by `_cartesian_to_wheels` in `sorca.py`. The result is saturated to $\pm v_{\text{max}}$ to respect the actuator limits.

11 The S-ORCA Algorithm

Algorithm 2 S-ORCA wheel-velocity selection

- 1: **Input:** $\mathbf{p}_i, \theta_i, (v_{l,i}, v_{r,i}), \mathbf{g}_i, r_i$ for all i ; parameters $\Delta t, v_{\max}, a_{\max}, \tau, d_{\max}, D_i, L_i$.
 - 2: Compute effective states $(\mathbf{p}_i^{\text{eff}}, \mathbf{v}_i^{\text{eff}}, R_i^{\text{eff}})$ by (47)–(52).
 - 3: Run standard ORCA (Part I) on the effective states to obtain $\mathbf{v}_i^{\text{eff}, \text{new}}$ for each robot.
 - 4: Convert to wheel velocities by (54).
 - 5: Saturate $v_{l,i}, v_{r,i}$ to $[-v_{\max}, v_{\max}]$.
 - 6: **return** $(v_{l,i}, v_{r,i})$ for all i .
-

11.1 Goals in the effective frame

ORCA computes the preferred velocity from the goal *in the effective frame*. Since the effective center is offset by D_i from the real center, the goal fed to ORCA must also be offset:

$$\mathbf{g}_i^{\text{eff}} = \mathbf{g}_i - D_i \begin{pmatrix} \cos \theta_i \\ \sin \theta_i \end{pmatrix}. \quad (56)$$

In `sorca.py` this offset is applied by the caller before invoking `compute_velocities`.

11.2 Choice of D

The parameter D_i controls the trade-off between *maneuverability* and *safety*:

- $D_i \rightarrow 0$: the effective center coincides with the real center, the robot behaves like a holonomic disk, but the inverse kinematics becomes ill-conditioned ($\det M_i \rightarrow 0$).
- $D_i \rightarrow \infty$: the effective center is far in front, the effective radius grows, and the robot becomes overly conservative.

The default $\kappa = 1$, $D_i = r_i$, gives $R_i^{\text{eff}} = 2r_i$ and $\det M_i = r_i/L_i$.

11.3 Relation to the real robot

The mapping (54) is exact for the *effective center*, not for the real center. The real robot's trajectory therefore lags the effective trajectory by a bounded error, which S-ORCA does not explicitly bound. This is the theoretical gap that NH-ORCA (Part III) fills.

12 Properties and Limitations of S-ORCA

Reciprocity. S-ORCA preserves the reciprocity of ORCA because every agent uses the same effective-center construction and the same responsibility factor $\frac{1}{2}$.

Collision guarantee. Collision-freeness is guaranteed for the *effective disks* of radius $R_i^{\text{eff}} = r_i + D_i$, not for the real disks. Because the real disk is contained in the effective disk, the real disks are also collision-free *provided the effective center is tracked exactly*. In practice, the tracking error introduces a small violation.

Computational cost. S-ORCA adds $O(N)$ work for the effective-state conversion and the inverse kinematics. The ORCA LP dominates the cost.

Part III: NH-ORCA (Non-Holonomic ORCA with Bounded Tracking Error)

13 Motivation

S-ORCA applies ORCA to a virtual effective center but does not bound the error between the effective trajectory and the real trajectory. NH-ORCA (Alonso-Mora, Breitenmoser, Rufli, Beardsley, and Siegwart, 2013) closes this gap by:

1. deriving a formal upper bound $\mathcal{E}$ on the tracking error of a differential-drive robot following a holonomic reference;
2. constructing the set S_{AHV} of holonomic velocities that can be tracked with error $\leq \mathcal{E}$;
3. defining a convex polygon $P_{AHV} \subseteq S_{AHV}$ inside which the holonomic velocity is projected;
4. applying ORCA to disks of radius $r_i + \mathcal{E}_i$, which are guaranteed to contain the real disks at all times.

The price is a more conservative behavior: the robots keep a larger distance because the effective radius now includes the tracking-error bound.

14 Tracking Error of a Differential-Drive Robot

14.1 Setup

Consider a differential-drive robot at the origin with heading $\theta = 0$. It is commanded to follow a holonomic reference velocity $\mathbf{v}_H = (V_H \cos \theta_H, V_H \sin \theta_H)$ for a maneuver time T , where $V_H \geq 0$ and $\theta_H \in (-\pi, \pi]$ is the direction of the reference relative to the robot's heading. Note that T is the *maneuver time* used to align the heading with the reference direction; it is distinct from the ORCA collision horizon τ of Part I. The robot executes the optimal constant controls (v, ω) over $[0, T]$. The tracking error is the distance between the ideal holonomic trajectory and the executed non-holonomic trajectory.

14.2 Derivation of the squared error

The ideal holonomic trajectory is

$$\mathbf{p}_H(t) = \mathbf{v}_H t = V_H t \begin{pmatrix} \cos \theta_H \\ \sin \theta_H \end{pmatrix}. \quad (57)$$

The executed trajectory under constant (v, ω) is

$$x(t) = \begin{cases} vt \cos(\omega t), & \omega \neq 0, \\ vt, & \omega = 0, \end{cases} \quad (58)$$

$$y(t) = \begin{cases} vt \sin(\omega t), & \omega \neq 0, \\ 0, & \omega = 0. \end{cases} \quad (59)$$

The squared tracking error at time T is

$$\varepsilon_H^2 = \|\mathbf{p}_H(T) - \mathbf{p}_{NH}(T)\|^2. \quad (60)$$

Expanding,

$$\varepsilon_H^2 = V_H^2 T^2 - \frac{2V_H T \sin \theta_H}{\omega} v + \frac{2(1 - \cos \theta_H)}{\omega^2} v^2. \quad (61)$$

This is Eq. (5) of Alonso-Mora et al. and the function `_tracking_error_sq` in `nhorca.py`. The limit $\omega \rightarrow 0$ is handled by L'Hôpital's rule, giving

$$\varepsilon_H^2 \xrightarrow{\omega \rightarrow 0} V_H^2 T^2 - 2V_H T^2 v + T^2 v^2. \quad (62)$$

In code this is implemented by substituting $\omega = \theta_H/T$ in the singular terms.

14.3 Optimal control for a given holonomic velocity

Given (V_H, θ_H) , the goal is to choose (v, ω) minimizing (61) subject to $|v| \leq v_{\max}$ and $|\omega| \leq \omega_{\max}$. Three regimes arise.

Region R_{A1} : $\omega = \theta_H/T \leq \omega_{\max}$. The angular velocity aligns the heading with the reference direction at time T . The optimal linear velocity is

$$v^* = V_H \frac{\theta_H \sin \theta_H}{2(1 - \cos \theta_H)}, \quad (63)$$

provided $|v^*| \leq v_{\max}$. Otherwise $v = \text{sign}(v^*)v_{\max}$ and we are in region R_{A2} .

Region R_{A2} : $|v^*| > v_{\max}$. The linear velocity saturates at $v = \text{sign}(v^*)v_{\max}$, and the resulting error is larger than the minimum.

Region R_B : $\theta_H/T > \omega_{\max}$. The heading cannot be aligned within T , so the optimal strategy is to rotate in place: $v = 0$, $\omega = \text{sign}(\theta_H)\omega_{\max}$.

The function `_optimal_control_for_holonomic` in `nhorca.py` implements this case analysis.

15 The Set S_{AHV} and Its Boundary

15.1 Definition

For a given maximum tracking error $\mathcal{E} > 0$ and maneuver time T , define

$$S_{AHV}(\mathcal{E}, T) = \left\{ \mathbf{v}_H \in \mathbb{R}^2 : \min_{v, \omega} \varepsilon_H(\mathbf{v}_H, v, \omega) \leq \mathcal{E} \right\}. \quad (64)$$

This is the set of holonomic velocities that can be tracked with error at most $\mathcal{E}$. It is a convex set containing the origin and symmetric under $\theta_H \mapsto -\theta_H$.

15.2 Maximum holonomic speed per direction

The boundary of S_{AHV} is characterized by the maximum speed $V_H^{\max}(\theta_H)$ along each direction. Alonso-Mora et al. derive (Theorem 2, Eq. 13) the following piecewise expression.

Case 1: $\theta_H/T \leq \omega_{\max}$. If

$$v_{\mathcal{E}}^* = \frac{\mathcal{E}}{T} \frac{\theta_H \sin \theta_H}{2(1 - \cos \theta_H)} \sqrt{\frac{2(1 - \cos \theta_H)}{2(1 - \cos \theta_H) - \sin^2 \theta_H}} \leq v_{\max}, \quad (65)$$

then

$$V_1 = \frac{\mathcal{E}}{T} \sqrt{\frac{2(1 - \cos \theta_H)}{2(1 - \cos \theta_H) - \sin^2 \theta_H}}, \quad V_H^{\max} = \min(V_1, v_{\max}). \quad (66)$$

Otherwise, the boundary is the smaller root of the quadratic $\alpha V_H^2 + \beta V_H + \gamma = 0$ with

$$\alpha = T^2, \quad \beta = -\frac{2T^2 \sin \theta_H}{\theta_H} v_{\max}, \quad \gamma = \frac{2T^2(1 - \cos \theta_H)}{\theta_H^2} v_{\max}^2 - \mathcal{E}^2. \quad (67)$$

Case 2: $\theta_H/T > \omega_{\max}$. The heading cannot be aligned, so the error is dominated by the in-place rotation:

$$V_3 = \frac{\mathcal{E} \omega_{\max}}{\theta_H}, \quad V_H^{\max} = \min(V_3, v_{\max}). \quad (68)$$

The function `_v_h_max_for_direction` implements this case analysis.

15.3 Polygon P_{AHV}

S_{AHV} is convex but its boundary (66)–(68) is transcendental, so the code approximates it by a convex polygon $P_{\text{AHV}} \subseteq S_{\text{AHV}}$ inscribed in the boundary. Two modes are provided.

Mode A (default): n -ray inscribed polygon. Sample n directions $\theta_k = -\pi + 2\pi k/n$, $k = 0, \dots, n-1$, and take the vertices

$$\mathbf{v}_k = V_H^{\max}(\theta_k) \begin{pmatrix} \cos \theta_k \\ \sin \theta_k \end{pmatrix}. \quad (69)$$

The polygon is the convex hull of these vertices, contained in S_{AHV} by construction.

Mode B: inscribed rectangle. Use the four axis directions $V_x = V_H^{\max}(0)$, $V_y = V_H^{\max}(\pi/2)$, and their negatives, giving the rectangle $[-V_x, V_x] \times [-V_y, V_y]$.

In both modes the polygon is expressed in the body frame and rotated to the world frame by

$$P_{\text{AHV}}(\theta_i) = R(\theta_i) P_{\text{AHV}}(0), \quad R(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}. \quad (70)$$

16 The NH-ORCA Algorithm

Algorithm 3 NH-ORCA wheel-velocity selection

- 1: **Input:** $\mathbf{p}_i, \theta_i, (v_{l,i}, v_{r,i}), \mathbf{g}_i, r_i$ for all i ; parameters $\Delta t, v_{\max}, \tau, d_{\max}, \mathcal{E}, T, \omega_{\max}$.
 - 2: **Adaptive error:** compute $\mathcal{E}_i \leq \mathcal{E}$ such that $r_i + r_j + \mathcal{E}_i + \mathcal{E}_j \leq \|\mathbf{p}_i - \mathbf{p}_j\|$ for all pairs (Remark 4).
 - 3: **Effective states:** $\mathbf{p}_i^{\text{eff}} = \mathbf{p}_i$, $R_i^{\text{eff}} = r_i + \mathcal{E}_i$, $\mathbf{v}_i^{\text{eff}} = (\dot{X}_i, \dot{Y}_i)$ from wheel speeds.
 - 4: **ORCA:** run standard ORCA on the effective disks to obtain $\mathbf{v}_i^{\text{eff}, \text{new}}$.
 - 5: **Projection:** for each i , build $P_{\text{AHV}}(\theta_i)$ with error $\mathcal{E}_i$, and project $\mathbf{v}_i^{\text{eff}, \text{new}}$ onto it by ray casting.
 - 6: **Mapping:** convert the projected velocity to (v_i, ω_i) by (63), then to wheel velocities by (45).
 - 7: **Saturation:** clip $v_{l,i}, v_{r,i}$ to $[-v_{\max}, v_{\max}]$.
 - 8: **return** $(v_{l,i}, v_{r,i})$ for all i .
-

16.1 Adaptive tracking error (Remark 4)

If two robots start very close, the expanded disks $r_i + \mathcal{E}_i$ and $r_j + \mathcal{E}_j$ may overlap even when the real disks do not. To prevent this, the code reduces $\mathcal{E}_i$ stepwise until

$$r_i + r_j + \mathcal{E}_i + \mathcal{E}_j \leq \|\mathbf{p}_i - \mathbf{p}_j\| \quad (71)$$

holds for every pair. The reduction is $\mathcal{E}_i \leftarrow \max(\mathcal{E}_{\min}, \mathcal{E}_i - 0.1\mathcal{E}_{\text{base}})$, applied to the larger of $\mathcal{E}_i, \mathcal{E}_j$ until (71) holds, with floor $\mathcal{E}_{\min} = 10^{-4}$. Because $\mathcal{E}_i$ only decreases, the actual tracking error remains bounded by the original $\mathcal{E}_{\text{base}}$.

16.2 Radial projection onto P_{AHV}

Given a velocity $\mathbf{v}$ outside P_{AHV} , the code computes the radial projection

$$\pi_P(\mathbf{v}) = \begin{cases} \mathbf{v}, & \mathbf{v} \in P_{\text{AHV}}, \\ t^* \frac{\mathbf{v}}{\|\mathbf{v}\|}, & \text{otherwise,} \end{cases} \quad (72)$$

where t^* is the smallest $t > 0$ such that $t\mathbf{v}/\|\mathbf{v}\|$ lies on the boundary of P_{AHV} . The boundary intersection is computed by testing each edge: for edge (a, b) with direction $e = b - a$, solve

$$a + ue = td, \quad u \in [0, 1], \quad t > 0, \quad (73)$$

where $d = \mathbf{v}/\|\mathbf{v}\|$. The solution is

$$u = \frac{(a \times d)_z}{(e \times d)_z}, \quad t = \frac{(a \times e)_z}{(e \times d)_z}, \quad (74)$$

and t^* is the smallest positive t among all edges. This is a radial (scaling) projection, not the orthogonal projection; it is a fast approximation of the QP solved in the paper, preserving the direction of the ORCA velocity.

16.3 Comparison with S-ORCA

	S-ORCA	NH-ORCA
Effective center	offset by D_i in front	real center
Effective radius	$r_i + D_i$	$r_i + \mathcal{E}_i$
Tracking error bound	not explicit	formal, $\mathcal{E}_i$
Tracking error adaptation	no	yes (Remark 4)
Projection set	none	P_{AHV} from $\mathcal{E}_i$
Inverse kinematics	M_i^{-1} from wheel base	(v, ω) from (63)
Guarantee	effective disks	real disks

Table 1: Comparison of S-ORCA and NH-ORCA.

Both algorithms reduce to standard ORCA in the holonomic limit ($D_i \rightarrow 0$ for S-ORCA, $\mathcal{E}_i \rightarrow 0$ for NH-ORCA).

17 Summary of the Extended Algorithm Family

We now have five algorithms, all built from the same geometric primitives derived in Part I:

1. **VO**: holonomic, one-sided; the forbidden set is a truncated cone.
2. **RVO**: holonomic, reciprocal; the forbidden set is a truncated cone with apex at $(\mathbf{v}_a + \mathbf{v}_b)/2$ and bounds scaled by $\frac{1}{2}$. Non-convex feasible set.
3. **ORCA** (`orca.py`): holonomic, reciprocal; the RVO cone is replaced by a single supporting half-plane, giving a convex QP. Responsibility factor $\frac{1}{2}$ (reciprocal) or 1 (static).
4. **S-ORCA** (`sorca.py`): differential-drive agents modeled as holonomic disks at an offset effective center; ORCA applied to the effective disks, then mapped to wheel velocities by M_i^{-1} .
5. **NH-ORCA** (`nhorca.py`): differential-drive agents modeled as holonomic disks of radius $r_i + \mathcal{E}_i$ at the real center; ORCA applied to the expanded disks, then projected onto P_{AHV} and mapped to (v, ω) with a formal tracking-error bound.

References

- [1] J. van den Berg, S. J. Guy, M. Lin, and D. Manocha, “Reciprocal n -body collision avoidance,” in *Robotics Research*, Springer, 2011, pp. 3–19.
- [2] J. van den Berg, M. Lin, and D. Manocha, “Reciprocal Velocity Obstacles for real-time multi-agent navigation,” in *IEEE ICRA*, 2008, pp. 1928–1935.
- [3] P. Fiorini and Z. Shiller, “Motion planning in dynamic environments using velocity obstacles,” *The International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, 1998.
- [4] S. J. Guy, J. van den Berg, M. Lin, and D. Manocha, “Clearpath: Highly parallel collision avoidance for multi-agent simulation,” in *ACM SIGGRAPH/Eurographics SCA*, 2009, pp. 177–187.
- [5] The RVO2 library, <https://gamma.cs.unc.edu/RVO2/>.
- [6] J. Snape, J. van den Berg, S. J. Guy, and D. Manocha, “The Hybrid Reciprocal Velocity Obstacle,” *IEEE Transactions on Robotics*, vol. 27, no. 4, pp. 696–706, 2011.
- [7] J. Snape, J. van den Berg, S. J. Guy, and D. Manocha, “Smooth and collision-free navigation for multiple robots under differential-drive constraints,” in *IEEE/RSJ IROS*, 2010, pp. 4580–4585.
- [8] J. Alonso-Mora, A. Breitenmoser, M. Rufli, P. Beardsley, and R. Siegwart, “Optimal Reciprocal Collision Avoidance for Multiple Non-Holonomic Robots,” in *Robotics Research: The 15th International Symposium ISRR*, Springer, 2013, pp. 203–216.